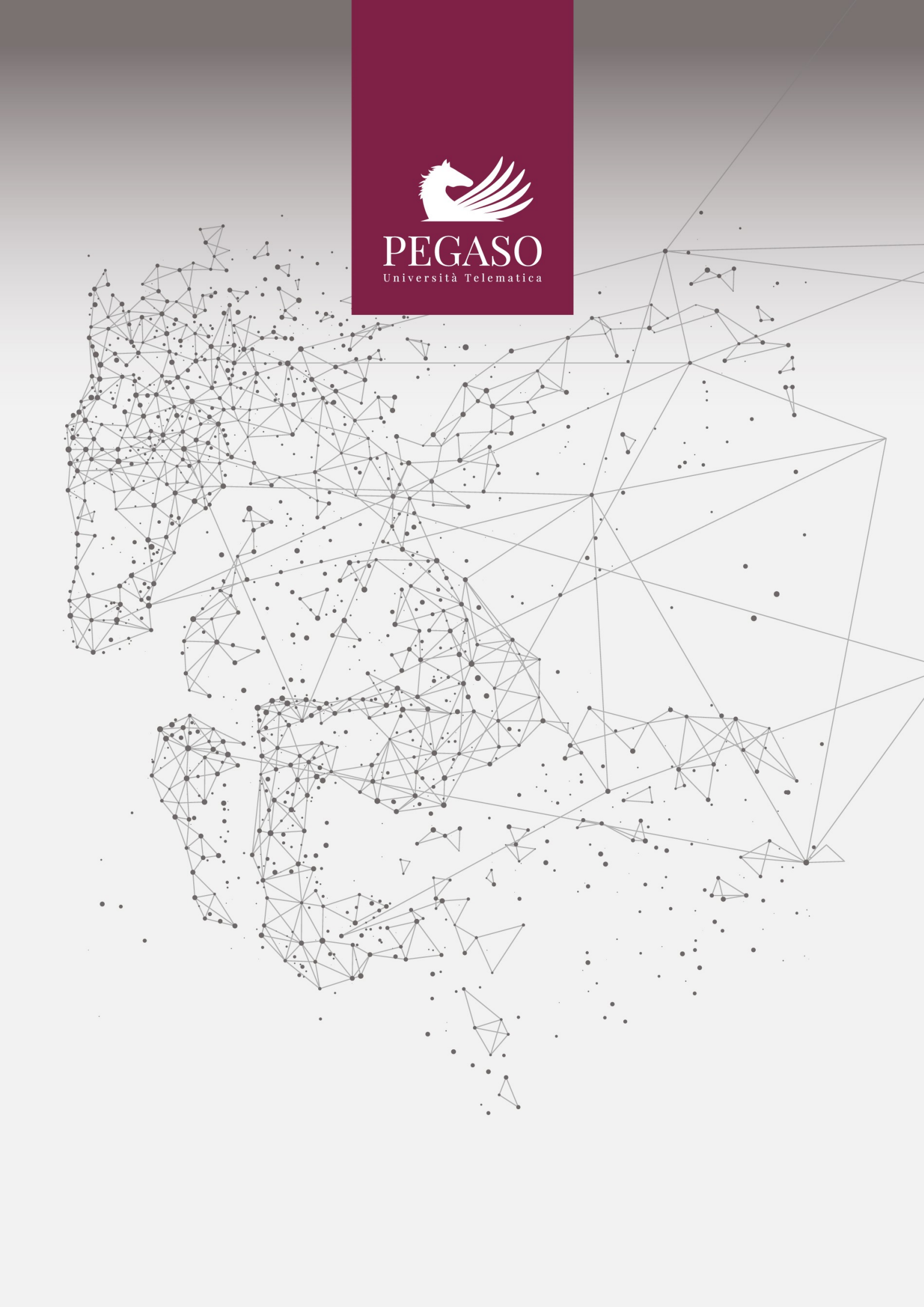

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. JQUERY 3
2. ESEMPIO BASE 5
3. SINTASSI 7
4. SELETTORI 8
5. EVENTI 9
6. CALLBACK 12
7. CHAINING 14
8. MANIPOLAZIONE DEL DOM 15
BIBLIOGRAFIA 16
SITOGRAFIA 17

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. JQuery

jQuery è una libreria JavaScript che semplifica la scrittura di codice in JavaScript, rendendo più facile e veloce lo sviluppo di applicazioni web. jQuery è stata creata nel 2006 da John Resig con l'obiettivo di semplificare la manipolazione del DOM (Document Object Model) e di fornire una sintassi più semplice per l'interazione con gli elementi HTML.

jQuery ha molte funzioni utili per gli sviluppatori web, come la selezione degli elementi HTML, la manipolazione del CSS, l'aggiunta di effetti animati, la gestione degli eventi, la gestione delle richieste AJAX e molto altro ancora.

jQuery è diventata una delle librerie JavaScript più popolari al mondo grazie alla sua facilità d'uso e alla vasta gamma di funzioni disponibili. Molti sviluppatori web preferiscono jQuery per la sua compatibilità multi-browser, la sua leggerezza, la sua documentazione completa e la sua comunità di supporto attiva.

In sintesi, jQuery è una libreria JavaScript molto utile che semplifica la scrittura di codice in JavaScript, consentendo di sviluppare applicazioni web più velocemente ed efficientemente.

Per includere jQuery nelle nostre pagine, possiamo scaricare la libreria dal sito ufficiale di jQuery (https://jquery.com/) oppure utilizzare un Content Delivery Network (CDN) che fornisce una copia di jQuery ospitata su un server remoto. Utilizzare un CDN è generalmente più veloce e affidabile, in quanto la copia di jQuery viene scaricata dal server CDN più vicino al client.

Ecco come includere jQuery utilizzando un CDN:

```
<!DOCTYPE html>
<html>
<head>
	<title>La mia pagina con jQuery</title>
	<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>
<body>
	<h1>La mia pagina con jQuery</h1>
	<p>Questa è la mia prima pagina con jQuery</p>
</body>
</html>
```

In questo esempio, abbiamo incluso la libreria jQuery tramite il tag script nel nostro documento HTML. L'URL specificato nel tag script corrisponde all'indirizzo del file JavaScript di jQuery sul CDN ufficiale di jQuery. Quando il browser scarica la pagina HTML, scaricherà automaticamente anche il file jQuery dal server remoto.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Una volta inclusa la libreria jQuery nella nostra pagina, possiamo iniziare a utilizzare le funzioni di jQuery per manipolare il nostro contenuto HTML.

Quando scarichiamo la libreria jQuery dal sito ufficiale, abbiamo la possibilità di scegliere tra diverse versioni, ognuna delle quali ha un uso specifico:

- **Compressed version (versione compressa)**: è la versione compressa di jQuery che occupa meno spazio sul disco rigido. Questa versione è ottimizzata per la distribuzione in produzione, in quanto il file è più piccolo e quindi viene scaricato più velocemente dai client.
- **Uncompressed version (versione non compressa)**: è la versione non compressa di jQuery che include tutti i commenti e la formattazione originale del codice. Questa versione è più facile da leggere e capire, in quanto il codice è ben strutturato e commentato. Tuttavia, il file è più grande rispetto alla versione compressa e quindi richiede più tempo per essere scaricato.
- **Minified version (versione minificata)**: è una versione compressa di jQuery che rimuove tutti i commenti e gli spazi vuoti dal codice. Questa versione è ottimizzata per la distribuzione in produzione, in quanto il file è ancora più piccolo rispetto alla versione compressa e quindi viene scaricato ancora più velocemente dai client.
- **Slim version (versione slim)**: è una versione ridotta di jQuery che include solo le funzioni più utilizzate. Questa versione è ideale per le applicazioni web che non richiedono l'utilizzo di tutte le funzioni di jQuery, in quanto il file è più piccolo e occupa meno spazio in memoria.

In sintesi, la differenza principale tra le varie versioni di jQuery sta nella dimensione del file e nella presenza di commenti e spazi vuoti. La versione compressa è ottimizzata per la distribuzione in produzione, mentre la versione non compressa è più facile da leggere e comprendere. La versione slim è invece indicata per applicazioni web che non richiedono tutte le funzioni di jQuery.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Esempio base

Ecco un esempio semplice di come utilizzare jQuery per modificare il testo di un elemento HTML:

```
<!DOCTYPE html>
<html>
<head>
	<title>Esempio jQuery</title>
	<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
	<script>
		$(document).ready(function() {
			$("h1").text("Questo è un titolo modificato con jQuery!");
		});
	</script>
</head>
<body>
	<h1>Questo è un titolo</h1>
	<p>Questo è un paragrafo</p>
</body>
</html>
```

In questo esempio, abbiamo incluso la libreria jQuery tramite il tag script nel nostro documento HTML.

Successivamente, abbiamo scritto una funzione jQuery che viene eseguita quando il documento è pronto. All'interno di questa funzione, abbiamo utilizzato il selettore $("h1") per selezionare l'elemento HTML <h1> e abbiamo utilizzato il metodo .text() per impostare il testo dell'elemento a "Questo è un titolo modificato con jQuery!".

Quando apriamo questa pagina HTML nel nostro browser, vedremo che il testo dell'elemento <h1> è stato modificato utilizzando jQuery.

Di seguito un altro esempio base:

```
<!DOCTYPE html>
<html>
<head>
	<title>La mia pagina con jQuery</title>
	<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
	<script>
		$(document).ready(function() {
			$("#mio_paragrafo").text("paragrafo modificato");
		});
	</script>
</head>
<body>
	<h1>La mia pagina con jQuery</h1>
	<p id="mio_paragrafo">Questo è il mio paragrafo</p>
</body>
</html>
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In questo esempio, abbiamo utilizzato il selettore $("#mio_paragrafo") per selezionare l'elemento HTML <p> con l'attributo id uguale a "mio_paragrafo". Successivamente, abbiamo utilizzato il metodo .text() per modificare il contenuto del paragrafo in "Questo è il mio paragrafo modificato con jQuery!".

Quando apriamo questa pagina HTML nel nostro browser, vedremo che il contenuto del paragrafo è stato modificato utilizzando jQuery. In questo esempio, abbiamo utilizzato un selettore di ID per selezionare un elemento specifico.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Sintassi

La sintassi di jQuery è progettata per semplificare la scrittura di codice JavaScript e rendere più facile l'interazione con gli elementi HTML; questa si compone di un selettore, un metodo e un eventuale valore. Ecco un esempio di sintassi di base:

```
$(selettore).metodo(valore);
```

- **$** è il simbolo che indica l'utilizzo della libreria jQuery.
- **selettore** è un'istruzione che indica l'elemento HTML su cui eseguire l'azione;
- **metodo** è l'azione che vogliamo eseguire sull'elemento selezionato;
- **valore** è un parametro opzionale che possiamo utilizzare per fornire ulteriori informazioni all'azione che stiamo eseguendo.

Ad esempio, se vogliamo selezionare tutti gli elementi <p> del nostro documento HTML e nasconderli, possiamo utilizzare la seguente sintassi:

```
$("p").hide();
```

In questo esempio, il selettore "p" seleziona tutti gli elementi <p> del nostro documento HTML, mentre il metodo .hide() nasconde gli elementi selezionati.

jQuery offre molti altri metodi per manipolare gli elementi HTML, come .show(), .text(), .html(), .attr(), .addClass(), .removeClass(), .toggleClass(), .animate(), .on(), .click(), e molti altri ancora.

Inoltre, jQuery supporta anche le funzioni **callback**, che ci consentono di eseguire azioni dopo che un'azione jQuery è stata completata; ad esempio, possiamo utilizzare la funzione .fadeIn() per mostrare gradualmente un elemento e utilizzare la funzione **callback** per eseguire un'azione dopo che l'elemento è stato mostrato completamente:

```
$("#mio_div").fadeIn(1000, function() {
  alert("L'elemento è stato mostrato completamente!");
});
```

In questo esempio, la funzione .fadeIn() viene eseguita con un tempo di durata di 1000 millisecondi (1 secondo) e la funzione **callback** viene eseguita quando l'elemento è stato mostrato completamente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Selettori

In jQuery, i selettori sono utilizzati per selezionare gli elementi HTML sui quali vogliamo eseguire un'azione. Esistono diversi tipi di selettori che ci consentono di selezionare gli elementi in base al loro tipo, classe, attributo, posizione e altro ancora. Ecco alcuni esempi di selettori jQuery:

- **Selettore di tipo**: seleziona tutti gli elementi di un determinato tipo (ad esempio, $("p") seleziona tutti gli elementi <p>).
- **Selettore di ID**: seleziona un elemento in base al suo attributo id (ad esempio, $("#mio_id") seleziona l'elemento con id uguale a "mio_id").
- **Selettore di classe**: seleziona tutti gli elementi con una determinata classe (ad esempio, $(".mialista") seleziona tutti gli elementi con la classe "mialista").
- **Selettore di attributo**: seleziona tutti gli elementi con un determinato attributo (ad esempio, $("[name='mio_nome']") seleziona tutti gli elementi con l'attributo name uguale a "mio_nome").
- **Selettore di figlio**: seleziona tutti gli elementi figli di un elemento specifico (ad esempio, $("ul li") seleziona tutti gli elementi <li> figli di un elemento <ul>).
- **Selettore di primo figlio**: seleziona il primo elemento figlio di un elemento specifico (ad esempio, $("ul li:first-child") seleziona il primo elemento <li> figlio di un elemento <ul>).
- **Selettore di ultimo figlio**: seleziona l'ultimo elemento figlio di un elemento specifico (ad esempio, $("ul li:last-child") seleziona l'ultimo elemento <li> figlio di un elemento <ul>).
- **Selettore di primo elemento**: seleziona il primo elemento di un determinato tipo (ad esempio, $("p:first") seleziona il primo elemento <p>).
- **Selettore di ultimo elemento**: seleziona l'ultimo elemento di un determinato tipo (ad esempio, $("p:last") seleziona l'ultimo elemento <p>).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Eventi

In jQuery, gli eventi sono azioni che vengono eseguite quando si verifica un'interazione dell'utente con un elemento HTML, come un clic del mouse, un hover, una pressione del tasto e molti altri ancora. Gli eventi in jQuery sono rappresentati da stringhe di testo che indicano l'azione da eseguire e vengono assegnati agli elementi HTML tramite il metodo .on(). Ecco alcuni esempi di eventi in jQuery:

- **click**: viene eseguito quando si fa clic su un elemento HTML
    - $(elemento).on("click", funzione)
- **hover**: viene eseguito quando si posiziona il mouse sopra o fuori da un elemento
    - $(elemento).on("mouseenter", funzione)
    - $(elemento).on("mouseleave", funzione)
- **keypress**: viene eseguito quando si preme un tasto sulla tastiera mentre l'elemento è in focus
    - $(elemento).on("keypress", funzione)
- **submit**: viene eseguito quando viene inviato un modulo
    - $(elemento).on("submit", funzione)
- **change**: viene eseguito quando il valore di un elemento cambia
    - $(elemento).on("change", funzione)
- **scroll**: viene eseguito quando l'utente scorre la pagina
    - $(elemento).on("scroll", funzione)
- **resize**: viene eseguito quando la finestra del browser viene ridimensionata
    - $(window).on("resize", funzione)

Ecco un esempio di come utilizzare l'evento click in jQuery per modificare il contenuto di un elemento HTML:

```
<!DOCTYPE html>
<html>
<head>
	<title>La mia pagina con jQuery</title>
	<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
	<script>
	$(document).ready(function() {
		$("#mio_bottone").on("click", function() {
			$("#mio_paragrafo").text("new text");
		});
	});
	</script>
</head>
<body>
	<h1>La mia pagina con jQuery</h1>
	<p id="mio_paragrafo">Questo è il mio paragrafo</p>
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
	<button id="mio_bottone">Cliccami!</button>
</body>
</html>
```

In questo esempio, abbiamo utilizzato il selettore "#mio_bottone" per selezionare l'elemento <button> con l'attributo id uguale a "mio_bottone". Successivamente, abbiamo utilizzato il metodo .on() per assegnare l'evento click all'elemento selezionato. Quando l'utente fa clic sul bottone, la funzione specificata viene eseguita e il contenuto dell'elemento con l'attributo id uguale a "mio_paragrafo" viene modificato.

Da notare che l'azione di modificare il contenuto di un elemento HTML in risposta all'evento click può essere realizzata anche con JavaScript nativo, senza l'utilizzo di jQuery.

Ecco un esempio di codice JavaScript che fa la stessa cosa dell'esempio precedente:

```
<!DOCTYPE html>
<html>
<head>
	<title>La mia pagina con JavaScript</title>
</head>
<body>
	<h1>La mia pagina con JavaScript</h1>
	<p id="mio_paragrafo">Questo è il mio paragrafo</p>
	<button id="mio_bottone">Cliccami!</button>
	<script>
	document.getElementById("mio_bottone").addEventListener("click",
		function() {
		document.getElementById("mio_paragrafo").innerHTML = "new text";
		});
	</script>
</body>
</html>
```

In molti casi, è possibile ottenere gli stessi risultati utilizzando JavaScript nativo al posto di jQuery. Tuttavia, ci sono alcune situazioni in cui l'utilizzo di jQuery può offrire alcuni vantaggi:

- **Sintassi semplificata**: La sintassi di jQuery è progettata per semplificare la scrittura di codice JavaScript e rendere più facile l'interazione con gli elementi HTML; ad esempio, utilizzando la sintassi di jQuery, possiamo selezionare un elemento HTML utilizzando un selettore simile a quello del CSS.
- **Compatibilità con i browser**: jQuery è stato progettato per funzionare su tutti i browser moderni, garantendo una maggiore compatibilità rispetto al JavaScript nativo.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- **Cross-browser**: jQuery fornisce una libreria di funzioni che funzionano su tutti i browser, nascondendo le differenze tra i vari browser e semplificando la scrittura di codice che funziona su tutte le piattaforme.
- **Ampia gamma di plugin**: jQuery è accompagnato da una vasta gamma di plugin che ci consentono di eseguire operazioni avanzate come animazioni, validazione dei moduli, richieste AJAX e molti altri.
- **Comunità attiva**: jQuery ha una grande comunità di sviluppatori che continuano a sviluppare nuovi plugin e rispondono alle domande degli utenti.

In generale, l'utilizzo di jQuery può rendere più facile e veloce lo sviluppo di applicazioni web, in particolare quando si lavora con elementi HTML complessi o quando si richiede una maggiore compatibilità con i browser. Tuttavia, dipende sempre dalle esigenze specifiche del progetto e dalla preferenza personale dello sviluppatore.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

6. Callback

Le callback sono funzioni che vengono passate come parametro ad un'altra funzione e che vengono eseguite in un secondo momento, solitamente dopo il completamento di un'operazione asincrona o di un'animazione.

In jQuery, le callback sono ampiamente utilizzate per gestire eventi e per eseguire operazioni dopo che una determinata azione è stata completata.

Ad esempio, possiamo utilizzare una callback con il metodo .fadeIn() per eseguire un'azione dopo che l'elemento è stato mostrato completamente. In questo caso, la callback verrà eseguita una volta che l'animazione del fadeIn sarà stata completata:

```
$("#mio_div").fadeIn(1000, function() {
  alert("L'elemento è stato mostrato completamente!");
});
```

In questo esempio, la funzione callback viene eseguita quando l'elemento con l'ID #mio_div è stato mostrato completamente. La funzione callback può essere qualsiasi funzione che vogliamo eseguire, ad esempio possiamo utilizzarla per modificare il contenuto di un altro elemento HTML o per effettuare una richiesta AJAX.

Inoltre, le callback possono essere annidate per eseguire più operazioni in sequenza. Ad esempio, possiamo utilizzare una callback con il metodo .animate() per eseguire una serie di animazioni in sequenza. In questo caso, la seconda animazione verrà eseguita solo dopo che la prima animazione sarà stata completata:

```
$("#mio_div").animate({left: '250px'}, 1000, function() {
  $("#mio_div").animate({top: '250px'}, 1000);
});
```

In questo esempio, la funzione callback viene utilizzata per eseguire una seconda animazione dopo che la prima è stata completata. La prima animazione sposta l'elemento con l'ID #mio_div verso destra di 250 pixel, mentre la seconda animazione sposta l'elemento verso il basso di altri 250 pixel.

Un esempio interessante di utilizzo delle callback in jQuery è il caricamento di immagini dinamiche in una pagina. In questo caso, le callback vengono utilizzate per eseguire operazioni quando le immagini sono state caricate correttamente o quando si verifica un errore nel caricamento.

Ad esempio, possiamo utilizzare una callback con il metodo .load() per eseguire un'azione quando l'immagine è stata caricata correttamente. In questo caso, la callback viene eseguita quando l'immagine è stata completamente caricata e mostrata nell'elemento

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
$("#mio_img").on("load", function() {
  alert("L'immagine è stata caricata correttamente!");
});
```

In questo esempio, la funzione callback viene eseguita quando l'immagine con l'ID #mio_img è stata caricata correttamente. Possiamo utilizzare questa callback per eseguire operazioni come mostrare un messaggio di successo o aggiornare l'interfaccia utente.

Allo stesso modo, possiamo utilizzare una callback con il metodo .error() per gestire gli errori di caricamento dell'immagine. In questo caso, la callback viene eseguita quando si verifica un errore nel caricamento dell'immagine:

```
$("#mio_img").on("error", function() {
  alert("Si è verificato un errore nel caricamento dell'immagine!");
});
```

In questo esempio, la funzione callback viene eseguita quando si verifica un errore nel caricamento dell'immagine con l'ID #mio_img. Possiamo utilizzare questa callback per gestire gli errori di caricamento dell'immagine e per aggiornare l'interfaccia utente in base all'errore.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

7. Chaining

Il chaining (o concatenazione) è una funzionalità di jQuery che ci consente di eseguire più metodi su un singolo elemento HTML in una singola istruzione. In pratica, anziché utilizzare una variabile per memorizzare l'elemento selezionato e quindi chiamare i metodi su quella variabile, possiamo concatenare i metodi direttamente sulla selezione jQuery stessa.

Ad esempio, supponiamo di voler selezionare un elemento con l'ID #mio_div, nasconderlo, modificarne il contenuto e poi visualizzarlo. Utilizzando il chaining, possiamo scrivere il seguente codice:

```
$("#mio_div")
  .hide()
  .html("Nuovo contenuto")
  .fadeIn(1000);
```

In questo esempio, stiamo selezionando l'elemento con l'ID #mio_div utilizzando il selettore jQuery $(). Successivamente, stiamo concatenando i metodi .hide(), .html() e .fadeIn() per nascondere l'elemento, modificarne il contenuto e quindi visualizzarlo con un'animazione di fade-in.

Il chaining ci consente di scrivere codice più leggibile e compatto, eliminando la necessità di memorizzare l'elemento selezionato in una variabile temporanea. Inoltre, il chaining ci consente di eseguire operazioni in sequenza su un singolo elemento, migliorando la comprensione del flusso di lavoro dell'applicazione.

Tuttavia, è importante ricordare che il chaining deve essere utilizzato con attenzione. Se concateniamo troppi metodi in una singola istruzione, il codice può diventare difficile da leggere e da comprendere. Inoltre, il chaining può rendere più difficile il debugging del codice, specialmente se si verificano errori in una delle operazioni concatenate.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

8. Manipolazione del DOM

Vi sono 3 metodi molto utili per la manipolazione del DOM:

- **text()**: il metodo .text() viene utilizzato per impostare o restituire il contenuto testuale degli elementi selezionati; ad esempio, possiamo utilizzare .text() per impostare il testo di un elemento o per restituire il testo dell'elemento selezionato.
- **html()**: il metodo .html() viene utilizzato per impostare o restituire il contenuto HTML degli elementi selezionati; ad esempio, possiamo utilizzare .html() per inserire del markup HTML all'interno di un elemento o per restituire il markup dell'elemento selezionato.
- **val()**: il metodo .val() viene utilizzato per impostare o restituire il valore dei campi di input di un form; ad esempio, possiamo utilizzare .val() per impostare il valore di un campo di input o per restituire il valore del campo di input selezionato.

Di seguito il codice:

```
// Impostiamo il testo di un elemento
$("p").text("Questo è un nuovo testo");
// Restituiamo il testo di un elemento
var testo = $("p").text();

// Impostiamo il contenuto HTML di un elemento
$("div").html("<p>Questo è un nuovo paragrafo</p>");
// Restituiamo il contenuto HTML di un elemento
var html = $("div").html();

// Impostiamo il valore di un campo di input
$("input[type='text']").val("Nuovo valore");
// Restituiamo il valore di un campo di input
var valore = $("input[type='text']").val();
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010.
- A. S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.
- Marco Andreini, Patrizia Andronico, Arianna Del Soldato, Cristian Lucchesi, Maurizio Martinelli, Giuseppe Vasarelli. Manuale corso di Tecnologie Web Istituto di Informatica e Telematica del CNR, 2005.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Sitografia

- HTML 4.01 Specification: http://www.w3.org/TR/html401/
- HTML5: http://www.w3.org/TR/html5/
- Javascript: JavaScript Tutorial (w3schools.com)
- HTML: http://www.w3schools.com/html/default.asp
- CSS: https://www.w3schools.com/css/default.asp
- CSS: http://www.w3.org/TR/CSS/

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).